

**CMP 142 – Object Oriented Programming
Fall 2024**

LAB-03

Issue Date: October 16, 2024

The objective of this lab is to:

1. Understand and practice classes, data hiding and abstraction concepts.
2. Practice good coding conventions e.g commenting, meaningful variable and functions names, properly indented and modular code.

Instructions!

1. This is a **graded** lab, you are strictly **NOT** allowed to discuss your solutions with your fellow colleagues, even not allowed asking how is he/she is doing, it may result in negative marking. You can **ONLY** discuss with your TAs or with your instructor.
2. Strictly follow good coding conventions (commenting, meaningful variable and functions names, properly indented and modular code.
3. Save your work frequently. Make a habit of pressing **CTRL+S** after every line of code you write.

Task 01:

[20 Marks]

You are required to create two C++ classes named Customer and Account to represent a customer with a bank account. The class specifications are as follows:

Class Structure:

Class Account:

- **Data Members:** double balance: //To store the current balance.

static double interestRate: //A static data member to store the interest rate static int

totalAccounts: //To keep track of the total number of Account objects created.

- **Constructors:**

A parameterized constructor that initializes the balance. It should also increment the totalAccounts each time a new account is created.

A copy constructor that creates a copy of an existing Account object. This should also increment the totalAccounts.

- **Static Member Functions:** static void setInterestRate(double rate): Sets the interest rate for all accounts.

static double getInterestRate(): Returns the current interest rate. static int

getTotalAccounts(): Returns the total number of Account objects created.

- **Other Functions:**

void deposit(double amount): Adds the specified amount to the balance.

void withdraw(double amount): Subtracts the specified amount from the balance, ensuring that the balance does not go negative. If amount is greater than the balance, print an appropriate message. void display()

const: Displays the balance and the current interest rate.

Class Customer:

- **Data Members:** string name: To store the customer's name.

Account account: An object of class Account to store the customer's account information.

- **Constructor:**

A parameterized constructor that initializes the name and account by taking parameters for both.

- **Member Functions:**

void display() const: Displays the customer's name and the details of their Account

Task:

Create two Customer objects using the parameterized constructor, each with an Account having an initial balance.

- Use the setInterestRate() function to set the interest rate to 5.0%.
- Display the details of each Customer .
- Create a third Customer object using the copy constructor of Account to duplicate the account of an existing customer.
- Make a deposit and a withdrawal for each customer's account.
- Display the total number of accounts and the current interest rate u

Task 02:

[20 Marks]

Create a **class named Teacher**. It contains the name of the teacher (a c-string containing at most 30 characters) and the teacher's office extension number (an integer) as **private member variables**. This class should NOT have a default constructor, and its only overloaded constructor should require the values of above two member variables as parameters.

Create a **class named Classroom**. It contains a room number (an integer) and the capacity (an integer) as **private member variables**. This class should NOT have a default constructor, and its only overloaded constructor should require values of both member variables as parameters.

Create a **class named Course**. A Course contains a course title (a c-string containing at most 40 characters), a Teacher, and a Classroom. Its **constructor** should require these 5 parameters: course title, the name and office extension number of the Teacher, and the room number and capacity of the Classroom.

Each of the above three classes should have a member function named **display()** which should display all attributes of a given object on screen.

Also create static variables CourseCount , ClassroomCount and TeacherCount that store total number of objects created of each respective class.

Also implement static functions **showCourseCount()** , **showClassroomCount()** and **showTeacherCount()** that displays them.

Also, you are allowed to implement getters and setters of any class according to your need.

Implement the **Copy constructor for each of the above three classes**.

Write a **main() function** that performs the following operations (in the given order):

1. First of all, create at least two different Course objects containing different values.
2. Then, your program should ask the user about all details of a course (including the
3. details of its Teacher and its Classroom) and create a Course object using the user-given input. 4. After that, your program should ask the user about all details of another course and
5. dynamically create a Course object (on the heap) using the user-given input.
6. Finally, your program should display the details of all created Courses on the screen.

Now, **implement another overloaded constructor** for the Course class. This constructor should take these 3 parameters: course title, a constant reference to a Teacher object, and a constant reference to a Classroom object. Then, **write a main() function to test the working of this 3-parameter overloaded constructor** as you did in the previous step.